

# Implementation vs. Technical Specification Analysis

---

**Analysis Date:** April 9, 2026

**Project:** Voice-of-Hands Sign Language Dataset Collection System

**Document Type:** Technical Compliance Assessment

---

## Executive Summary

---

This document provides a comprehensive comparison between the current Voice-of-Hands implementation and the Technical Developer Specification for SSL (Sri Lankan Sign Language) snippet creation pipeline. The analysis evaluates compliance across six development phases and identifies gaps, strengths, and recommendations.

**Overall Compliance Score: ~53%**

---

## Table of Contents

---

1. [Phase 1: Spatial Preprocessing](#)
  2. [Phase 2: Temporal Segmentation](#)
  3. [Phase 3: Handling Synchronization and Decalage](#)
  4. [Phase 4: Synchronous Audio Extraction](#)
  5. [Phase 5: Skeletal Feature Extraction](#)
  6. [Phase 6: Deferred Post-Processing](#)
  7. [Compliance Summary](#)
  8. [Recommendations](#)
-

# Phase 1: Spatial Preprocessing (PiP Extraction)

## Specification Requirements

### From Technical Spec:

*The sign language interpreter appears in a Picture-in-Picture (PiP) window, usually in the bottom-right corner.*

- **Define ROI Coordinates:** Define static coordinates to crop the PiP window from the 1080p source
- **Resolution:** A standard 224×224 or 512×512 crop is recommended for 3D-CNN backbones
- **Normalization:** Convert frames to grayscale or apply contrast enhancement to mitigate motion blur during rapid hand movements

## Current Implementation

| Aspect            | Specification      | Implementation                  | Status      |
|-------------------|--------------------|---------------------------------|-------------|
| ROI Detection     | Static coordinates | Dynamic border detection (auto) | ✔ BETTER    |
| Output Resolution | 224×224 or 512×512 | 256×256 (default, configurable) | ✔ COMPLIANT |
| Normalization     | Grayscale/contrast | Grayscale for detection only    | ⚠ PARTIAL   |

## Code Evidence

### ROI Detection (sli\_detector.py):

```
class SLIDetector:
    def __init__(self, video_path: str, border_margin: float = 0.15):
        # Dynamic border detection instead of static coordinates
        self.border_margin = border_margin
        # Automatically detects light-colored borders around interpreter
```

### Resolution Options (sli\_detector.py, timestamp\_extractor.py):

```
# Configurable output size
output_size: tuple = (256, 256) # Default
target_size: Optional[tuple] = (256, 256) # DNN training

# Also supports: (128, 128) or "original"
```

### Grayscale Usage (sli\_detector.py - Lines 274, 285):

```
# Used for detection algorithms, not applied to output
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Output clips remain in RGB color space
```

## Assessment

### Strengths:

-  Superior to specification - Dynamic border detection is more robust than static coordinates
-  Handles various PiP positions automatically (bottom-right, bottom-left, etc.)
-  Resolution within recommended range (256×256 between 224 and 512)
-  Configurable for different model requirements

### Gaps:

-  Final output clips are RGB, not grayscale normalized
-  No contrast enhancement applied to mitigate motion blur
-  Normalization only used internally for detection algorithms

**Compliance: 80% - Good detection, missing output normalization**

# Phase 2: Temporal Segmentation

## Specification Requirements

### From Technical Spec:

*The duration must encapsulate the three essential phases of sign production: preparation, stroke, and retraction.*

- **Continuous Sentence Window:** 5.0 to 11.0 seconds (CSLR for complex grammar)
- **Isolated Word Window:** 3.0 seconds (SSL400 standard for fingerspelling)
- **Raw Extraction Window:** 10.0 to 15.0 seconds (buffer for interpreter lag)

## Current Implementation

| Window Type | Specification       | Implementation              | Status      |
|-------------|---------------------|-----------------------------|-------------|
| Continuous  | 5.0 - 11.0 seconds  | 5.0s (default)              | ✓ COMPLIANT |
| Isolated    | 3.0 seconds         | Configurable (can set 3.0s) | ✓ COMPLIANT |
| Raw Buffer  | 10.0 - 15.0 seconds | Not implemented             | ✗ MISSING   |

## Code Evidence

### Default 5-Second Clips:

```
# sli_detector.py - Line 886
def extract_sli_clips(self, result: DetectionResult, output_dir: str,
                      clip_duration: float = 5.0, # Default matches spec
                      overlap: float = 0.5, ...):

# timestamp_extractor.py - Line 61
def extract_clips_with_metadata(self, crop_region: Dict[str, int],
                               clip_duration: float = 5.0, ...):
```

### Configurable Duration:

```
# Can be adjusted for different use cases
clips = detector.extract_sli_clips(result, "output/", clip_duration=3.0) # Iso
clips = detector.extract_sli_clips(result, "output/", clip_duration=5.0) # Con
clips = detector.extract_sli_clips(result, "output/", clip_duration=10.0) # Long
```

## Overlap Support:

```
# Provides temporal context across boundaries
step_frames = int(frames_per_clip * (1 - overlap))
# Default 50% overlap helps capture phrase boundaries
```

## Assessment

### Strengths:

- ✓ 5-second default matches specification's continuous window lower bound
- ✓ Fully configurable for different temporal requirements
- ✓ Overlap strategy helps with boundary effects
- ✓ Validated with research (see 5-Second\_Clip\_Duration\_Strategy.md)

### Gaps:

- ✗ No dedicated 10-15 second "raw extraction window" implementation
- ✗ Missing prosodic buffer concept from specification
- ⚠ No hierarchical segmentation approach (spec recommends 10-15s raw, then segment)

### Recommended Architecture (Not Implemented):

1. Extract 10-15s RAW clips (with prosodic window)
2. Then segment into 5s clips for training
3. Maintain relationship between raw and processed

### Compliance: 60% - Basic segmentation works, missing buffer strategy

# Phase 3: Handling Synchronization and Decalage

## Specification Requirements

### From Technical Spec:

A major challenge in broadcast data is **Decalage (Interpreter Lag)**—the delay between the spoken word and the signed rendition. For SSL, the average lag is approximately **3.36 seconds**.

**Implementation:** When clipping, apply a temporal offset. Maintain a buffer of at least **2.0s pre-audio** and **5.0s post-audio** to ensure the signed message is not truncated.

## Current Implementation

| Requirement       | Specification      | Implementation  | Status      |
|-------------------|--------------------|-----------------|-------------|
| Lag Compensation  | 3.36s offset       | Not implemented | ✗ MISSING   |
| Pre-audio Buffer  | 2.0s before        | Not implemented | ✗ MISSING   |
| Post-audio Buffer | 5.0s after         | Not implemented | ✗ MISSING   |
| Lag Awareness     | Document/calculate | Documented only | ⚠ AWARENESS |

## Code Evidence

### Current Synchronous Extraction (timestamp\_extractor.py):

```
# Line 380 - No lag offset applied
for i, clip_file in enumerate(clip_files):
    start_time = i * clip_duration # ❌ No lag compensation
    end_time = start_time + clip_duration

# Extract audio at same timestamp as video
subprocess.run([
    'ffmpeg', '-ss', str(start_time), # Same timing, no offset
    '-t', str(clip_duration), ...
])
```

## Documentation Shows Awareness (AUDIO\_SIGN\_ALIGNMENT\_METHODODOLOGY.md):

```
### 2.1 Interpreter Translation Lag
Sign language interpreters typically lag behind speech by 2-5 seconds.

Sign language interpreters in broadcast settings typically exhibit a
2-5 second lag behind spoken content
```

## But No Implementation:

```
# What SHOULD be implemented (not present):
LAG_OFFSET = 3.36 # seconds

# For video clip
sign_start = audio_start + LAG_OFFSET
sign_end = audio_end + LAG_OFFSET

# With buffers
clip_start = audio_start - 2.0 # Pre-buffer
clip_end = audio_end + 5.0 # Post-buffer
```

## Assessment

### Strengths:

- ☒ Project documentation recognizes the lag issue
- ☒ Research notes mention 2-5 second delay pattern
- ☒ Metadata structure supports timestamp tracking

**Critical Gaps:**

-  **No temporal offset applied during extraction**
-  **Audio and video extracted at identical timestamps**
-  **Risk of speech-sign misalignment in training data**
-  **No buffer windows to ensure complete gestures**

**Impact:** This is the **most significant gap**. Without lag compensation, the dataset may contain:

- Speech from time T aligned with signs from time T (incorrect)
- Should be: Speech from time T aligned with signs from time T+3.36s

**Example Misalignment:**

Current Implementation:

Audio Clip [0s-5s]: "පාර්ලිමේන්තු සභාව ආරම්භ වේ"

Video Clip [0s-5s]: [Interpreter signing PREVIOUS sentence from -3.36s]

 MISALIGNED

Spec Requirement:

Audio Clip [0s-5s]: "පාර්ලිමේන්තු සභාව ආරම්භ වේ"

Video Clip [3.36s-8.36s]: [Interpreter signing THIS sentence]

 ALIGNED

**Compliance: 10% - Documented but not implemented**

## Phase 4: Synchronous Audio Extraction

---

### Specification Requirements

#### From Technical Spec:

*Audio must be extracted immediately during the snippet creation phase to preserve the temporal relationship with the visual signs.*

**Target Format:** Mono, 16,000Hz (16kHz), 16-bit PCM .wav file

#### **FFmpeg Implementation:**

```
bash ffmpeg -i input_snippet.mp4 -vn -acodec pcm_s16le -ac 1 -ar 16000
output_audio.wav
```

Current Implementation

| Requirement       | Specification           | Implementation          | Status                          |
|-------------------|-------------------------|-------------------------|---------------------------------|
| Format            | Mono, 16kHz, 16-bit PCM | Exact match             | <div>✓</div> <div>PERFECT</div> |
| Timing            | During snippet creation | Simultaneous extraction | <div>✓</div> <div>PERFECT</div> |
| FFmpeg Parameters | Exact command           | Matches specification   | <div>✓</div> <div>PERFECT</div> |

Code Evidence

Audio Extraction (timestamp\_extractor.py - Line 243):

```
def _extract_audio_clip(self, start_time: float, duration: float, output_path: str):
    """Extract audio clip using ffmpeg"""
    try:
        subprocess.run([
            'ffmpeg', '-y', '-loglevel', 'error',
            '-i', self.video_path,
            '-ss', str(start_time),
            '-t', str(duration),
            '-vn', # ✓ Disables video stream
            '-acodec', 'pcm_s16le', # ✓ 16-bit PCM encoding
            '-ar', '16000', # ✓ 16kHz sample rate
            '-ac', '1', # ✓ Mono channel
            output_path
        ], check=True, timeout=30)
```

Also in run\_audio\_extraction.py (Line 74):

```
subprocess.run([
    'ffmpeg', '-y', '-loglevel', 'error',
    '-i', original_video,
    '-ss', str(start_time),
    '-t', str(clip_duration),
    '-vn', # No video
    '-acodec', 'pcm_s16le', # PCM 16-bit ✓
    '-ar', '16000', # 16kHz (optimal for ASR) ✓
    '-ac', '1', # Mono ✓
    audio_path
])
```

### Synchronous Extraction (timestamp\_extractor.py - Line 148):

```
# Extract audio immediately during video clip creation
if extract_audio:
    self._extract_audio_clip(clip_start_time, clip_duration, audio_path)
```

## Assessment

### Strengths:

- ✓ **Perfect implementation** - Matches specification exactly
- ✓ FFmpeg parameters identical to spec requirements
- ✓ Audio extracted synchronously with video clips
- ✓ Optimal format for ASR (Whisper compatibility)
- ✓ Proper error handling and timeout protection

### No Gaps Identified

### Parameter Validation:

|                 |                             |   |
|-----------------|-----------------------------|---|
| Spec: -vn       | → Implementation: -vn       | ✓ |
| Spec: pcm_s16le | → Implementation: pcm_s16le | ✓ |
| Spec: -ac 1     | → Implementation: -ac 1     | ✓ |
| Spec: -ar 16000 | → Implementation: -ar 16000 | ✓ |

**Compliance: 100% ✓ - Perfect implementation**

# Phase 5: Skeletal Feature Extraction

## Specification Requirements

### From Technical Spec:

- To ignore complex parliamentary backgrounds, extract mathematical skeletal data.
- **Landmark Selection:** Use MediaPipe Holistic to extract landmarks
  - **Reduce 532 3D landmarks to 85 points:**
    - 21 per hand (42 total)
    - 6 for upper body
    - 37 for facial expressions
  - **Uniform Sampling:** Select K=30 frames evenly spaced across the window
  - **Purpose:** Handle visual complexity of parliament chamber

## Current Implementation

| Requirement         | Specification        | Implementation      | Status       |
|---------------------|----------------------|---------------------|--------------|
| MediaPipe Usage     | Holistic landmarks   | Pose detection only | ⚠️ PARTIAL   |
| Landmark Extraction | 85-point subset      | Not extracted       | ❌ MISSING    |
| Frame Sampling      | K=30 uniform frames  | Not implemented     | ❌ MISSING    |
| Output Format       | Skeletal data files  | Video clips only    | ❌ MISSING    |
| Background Handling | Skeletal abstraction | Crops video         | ⚠️ DIFFERENT |

## Code Evidence

MediaPipe Available (sli\_detector.py - Line 389-395):

```
# MediaPipe used for DETECTION, not feature extraction
def _detect_pose(self, sample_frames: int, start_time: float = 0):
    try:
        import mediapipe as mp
    except ImportError:
        print(" [Warning] MediaPipe not installed")
        return self._fallback_detection("pose")

    mp_pose = mp.solutions.pose # Only POSE, not HOLISTIC
    pose = mp_pose.Pose(...)
```

### Landmarks Used for Detection Only:

```
# Line 419
results = pose.process(rgb)

if results.pose_landmarks:
    landmarks = results.pose_landmarks.landmark

    # Used to calculate bounding box, NOT saved
    x_coords = [lm.x * width for lm in landmarks]
    y_coords = [lm.y * height for lm in landmarks]
    bbox = (min(x_coords), min(y_coords), max(x_coords), max(y_coords))

    # ✗ Landmarks discarded after detection
    # ✗ Not saved for training
```

### Output is RGB Video, Not Skeletal Data:

```
# timestamp_extractor.py - Saves video frames
resized = cv2.resize(cropped, output_size, interpolation=cv2.INTER_CUBIC)
out.write(resized) # ✗ Saves RGB pixels, not landmarks
```

## What SHOULD Be Implemented

### According to Specification:

```
# Not implemented - Example of what's needed:
import mediapipe as mp

mp_holistic = mp.solutions.holistic
holistic = mp_holistic.Holistic()

# Process video
results = holistic.process(frame)

# Extract 85-point subset
landmarks = {
    'left_hand': results.left_hand_landmarks[:21],    # 21 points
    'right_hand': results.right_hand_landmarks[:21],  # 21 points
    'pose': [results.pose_landmarks[i] for i in [0,2,5,7,8,11]], # 6 points
    'face': results.face_landmarks[:37] # 37 points
}

# Uniform sampling: K=30 frames
sampled_indices = np.linspace(0, total_frames-1, 30, dtype=int)

# Save as numpy array or JSON
np.save('clip_landmarks.npy', landmark_array)
```

## Assessment

### Strengths:

-  MediaPipe library installed and functional
-  Pose detection working for ROI finding
-  Infrastructure exists to extend to Holistic

### Critical Gaps:

-  Uses Pose instead of Holistic (missing hand/face details)
-  Landmarks not saved or exported
-  No 85-point subset extraction
-  No K=30 uniform frame sampling
-  Outputs RGB video instead of skeletal data
-  Cannot handle complex backgrounds as spec intends

**Impact:** The specification's approach with skeletal features is designed to:

1. Make models invariant to background (parliament chamber complexity)

2. Reduce data size (85 floats vs 256×256×3 RGB)
3. Focus on motion dynamics (mathematical representation)

Your video-based approach works but is:

- More sensitive to background variations
- Larger file sizes
- May struggle with complex parliamentary settings (though cropping helps)

**Compliance: 20% - MediaPipe available but not used for feature extraction**

## Phase 6: Deferred Post-Processing (Transcription & Alignment)

### Specification Requirements

From Technical Spec:

*The transcription phase is performed later as an asynchronous task once the raw snippet library is established.*

- **Automated Transcription:** Process .wav files using fine-tuned Whisper model
- **Grammatical Reordering:** Sinhala SVO → SSL SOV structure
- **Weakly Supervised Alignment:** Use CTC loss to learn sign boundaries without frame-level annotation

### Current Implementation

| Requirement     | Specification      | Implementation      | Status    |
|-----------------|--------------------|---------------------|-----------|
| Whisper ASR     | Fine-tuned Whisper | Whisper (medium)    | ✓ PERFECT |
| Word Timestamps | Required           | Extracted & saved   | ✓ PERFECT |
| Sinhala Support | Language handling  | Full support ('si') | ✓ PERFECT |

| Requirement            | Specification      | Implementation    | Status      |
|------------------------|--------------------|-------------------|-------------|
| SVO → SOV Reordering   | Grammar conversion | Not implemented   | ✗ MISSING   |
| CTC Alignment          | Weakly supervised  | Not implemented   | ✗ MISSING   |
| Frame-level Annotation | Avoid manual work  | Not needed (good) | ✓ COMPLIANT |

Code Evidence

Whisper Transcription (timestamp\_extractor.py - Line 254-330):

```
def transcribe_audio_clips(self, metadata_path: str, model_name: str = "base",
                           language: str = None) -> Dict:
    """Transcribe all audio clips using Whisper"""
    import whisper

    # ✅ Load Whisper model
    model = whisper.load_model(model_name)

    # ✅ Transcribe with word-level timestamps
    if language:
        result = model.transcribe(audio_file,
                                   language=language,      # ✅ Sinhala support
                                   word_timestamps=True)    # ✅ Word-level timing

    # ✅ Extract confidence scores
    if 'segments' in result and result['segments']:
        for segment in result['segments']:
            if 'no_speech_prob' in segment:
                confidences.append(1.0 - segment['no_speech_prob'])

    # ✅ Word-level details
    if 'words' in segment:
        for word in segment['words']:
            word_details.append({
                'word': word.get('word', '').strip(),
                'start': round(word.get('start', 0), 3),
                'end': round(word.get('end', 0), 3),
                'probability': round(word.get('probability', 0), 3)
            })
```

### Metadata Saved (alignment\_metadata.json):

```
{
  "transcription": "පාර්ලිමේන්තු සභාව කරුණු සලකා බලයි",
  "transcription_language": "si",
  "transcription_confidence": 0.85,
  "word_timestamps": [
    {"word": "පාර්ලිමේන්තු", "start": 0.2, "end": 1.1, "probability": 0.92},
    {"word": "සභාව", "start": 1.2, "end": 1.8, "probability": 0.88}
  ]
}
```

### What's MISSING - SVO → SOV Reordering:

```
# Not implemented - Example of what's needed:
def reorder_sinhala_to_ssl(text: str) -> str:
    """Convert Sinhala SVO grammar to SSL SOV grammar"""
    # Use POS tagger to identify Subject, Verb, Object
    # Rearrange to SSL signing order
    # Example: "මම කෑම කනවා" (I food eat) → "මම කෑම කනවා" (I food eat)
    # (Sinhala already has flexible word order, but sign language differs)
    pass
```

### What's MISSING - CTC Alignment:

```
# Not implemented - Weakly supervised alignment
import torch
import torch.nn as nn

# CTC loss for boundary detection
ctc_loss = nn.CTCLoss()

# Learn alignment between:
# - Audio transcript (character/phoneme sequence)
# - Video frames (sign sequence)
# Without requiring frame-by-frame labels
```

## Assessment

### Strengths:

-  **Excellent ASR implementation** - Whisper integration is production-ready

-  Word-level timestamps extracted and saved
-  Confidence scoring tracked
-  Sinhala language fully supported
-  Metadata structure well-designed for research

**Gaps:**

-  No grammatical reordering (SVO → SOV) for sign language structure
-  No CTC-based alignment implementation
-  Missing linguistic preprocessing step
-  Assumption that speech order matches sign order (not always true)

**Impact:** The specification expects linguistic awareness:

- Spoken Sinhala may follow different word order than SSL
- CTC alignment would automatically learn sign boundaries
- Current approach assumes temporal synchronization (which is broken by Phase 3 gap)

**Compliance: 50% - ASR perfect, missing linguistic processing & CTC**

# Compliance Summary

## Overall Assessment by Phase

| Phase   | Component                | Spec Alignment             | Score | Status                                                                                                |
|---------|--------------------------|----------------------------|-------|-------------------------------------------------------------------------------------------------------|
| Phase 1 | PiP Extraction           | Better than spec (dynamic) | 80%   |  Good            |
| Phase 2 | Temporal Segmentation    | Basic implementation       | 60%   |  Partial         |
| Phase 3 | <b>Decalage Handling</b> | <b>Not implemented</b>     | 10%   |  <b>Critical</b> |
| Phase 4 | Audio Extraction         | Perfect match              | 100%  |  Perfect         |

| Phase   | Component         | Spec Alignment                 | Score | Status    |
|---------|-------------------|--------------------------------|-------|-----------|
| Phase 5 | Skeletal Features | Only detection, not extraction | 20%   | ✗ Missing |
| Phase 6 | Post-Processing   | ASR perfect, no CTC/reordering | 50%   | ⚠ Partial |
|         |                   | Overall Average                | 53%   |           |

Feature Matrix

✔ Implemented & Matching Spec (100%)

- Audio extraction (mono 16kHz PCM)
- Whisper ASR transcription
- Word-level timestamps
- Sinhala language support
- Metadata JSON structure

⚠ Partially Implemented (50-80%)

- PiP detection (better than spec, but missing normalization)
- Temporal segmentation (5s clips work, missing buffer approach)
- Post-processing (ASR perfect, missing linguistic components)

✗ Missing Critical Features (0-20%)

- **Interpreter lag compensation** (3.36s offset)
- **Pre/post audio buffers** (2s/5s)
- **Skeletal feature extraction** (85-point landmarks)
- **K=30 frame sampling**
- **SVO → SOV grammatical reordering**
- **CTC-based weakly supervised alignment**

- **Raw 10-15s extraction windows**

---

## Detailed Gap Analysis

---

### Critical Priority (Breaks Core Functionality)

#### 1. No Decalage (Lag) Compensation ●

**Impact:** HIGH - Breaks speech-sign alignment

**Current State:**

```
# Audio and video extracted at same timestamp
start_time = i * clip_duration
audio_clip = extract_audio(start_time, duration)
video_clip = extract_video(start_time, duration)
```

**Required State:**

```
# Should offset by ~3.36 seconds
INTERPRETER_LAG = 3.36

audio_start = i * clip_duration
video_start = audio_start + INTERPRETER_LAG

# With buffers
audio_clip = extract_audio(audio_start - 2.0, duration + 2.0 + 5.0)
video_clip = extract_video(video_start, duration)
```

**Consequence:** Training data has misaligned speech-sign pairs, may degrade model performance.

#### 2. No Skeletal Feature Extraction ●

**Impact:** MEDIUM-HIGH - Limits background invariance

**Current State:**

```
# Outputs RGB video frames  
output: 256×256×3 RGB pixels × 150 frames = ~28MB per 5s clip
```

### Required State:

```
# Should output skeletal landmarks  
output: 85 landmarks × 3 coordinates × 30 frames = ~10KB per 5s clip  
# 2800× smaller, background-invariant
```

**Consequence:** Models are background-dependent, larger dataset size, may struggle with parliament complexity.

## High Priority (Reduces Effectiveness)

### 3. No CTC Alignment 🟡

**Impact:** MEDIUM - Limits weakly supervised learning

**Required for:** Learning sign boundaries automatically without expensive frame-level annotation.

### 4. No Grammatical Reordering 🟡

**Impact:** MEDIUM - Linguistic misalignment

**Required for:** Handling SVO (spoken) vs SOV (signed) structural differences.

## Medium Priority (Specification Refinements)

### 5. No Buffer Windows 🟡

- Missing 10-15s raw extraction
- Missing 2s pre-audio buffer
- Missing 5s post-audio buffer

### 6. No Output Normalization 🟡

- RGB clips not grayscale normalized
- No contrast enhancement for motion blur

# Recommendations

---

## Immediate Actions (Critical Fixes)

### 1. Implement Lag Compensation

#### Code Addition Needed:

```
# Add to timestamp_extractor.py
class TimestampedClipExtractor:
    def __init__(self, video_path: str, output_base_dir: str,
                  interpreter_lag: float = 3.36): # Add this parameter
        self.interpreter_lag = interpreter_lag

    def extract_clips_with_metadata(self, ...):
        # When extracting clips
        audio_start = clip_index * clip_duration
        audio_end = audio_start + clip_duration

        # Offset video by lag
        video_start = audio_start + self.interpreter_lag
        video_end = audio_end + self.interpreter_lag

        # Extract with buffers
        buffered_start = audio_start - 2.0 # Pre-buffer
        buffered_end = audio_end + 5.0     # Post-buffer
```

#### Validation:

```
# Check alignment quality
def validate_alignment(metadata):
    for clip in metadata['clips']:
        audio_time = clip['audio_timestamp']
        video_time = clip['video_timestamp']
        lag = video_time - audio_time
        assert abs(lag - 3.36) < 0.5, f"Lag {lag} outside expected range"
```

## 2. Add Skeletal Feature Extraction

### Code Addition Needed:

```
# New module: skeletal_extractor.py
import mediapipe as mp
import numpy as np

class SkeletalFeatureExtractor:
    def __init__(self):
        self.holistic = mp.solutions.holistic.Holistic(
            static_image_mode=False,
            model_complexity=1
        )

    def extract_85_landmarks(self, video_path, output_path, k_frames=30):
        """Extract 85-point skeletal features"""
        cap = cv2.VideoCapture(video_path)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

        # Uniform sampling: K=30 frames
        sample_indices = np.linspace(0, total_frames-1, k_frames, dtype=int)

        landmarks_sequence = []

        for idx in sample_indices:
            cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
            ret, frame = cap.read()

            results = self.holistic.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

            # Extract 85 points
            points_85 = self._extract_subset(results)
            landmarks_sequence.append(points_85)

        # Save as numpy array
        np.save(output_path, np.array(landmarks_sequence))

    def _extract_subset(self, results):
        """Extract 21+21+6+37 = 85 points"""
        points = []

        # Left hand: 21 points
        if results.left_hand_landmarks:
            points.extend([(lm.x, lm.y, lm.z) for lm in results.left_hand_landmarks])
```

```
else:
    points.extend([(0,0,0)] * 21)

# Right hand: 21 points
if results.right_hand_landmarks:
    points.extend([(lm.x, lm.y, lm.z) for lm in results.right_hand_landmarks])
else:
    points.extend([(0,0,0)] * 21)

# Upper body: 6 points (shoulders, elbows, wrists)
if results.pose_landmarks:
    body_indices = [11, 12, 13, 14, 15, 16] # Key upper body points
    points.extend([(results.pose_landmarks.landmark[i].x,
                    results.pose_landmarks.landmark[i].y,
                    results.pose_landmarks.landmark[i].z) for i in body_indices])
else:
    points.extend([(0,0,0)] * 6)

# Face: 37 points
if results.face_landmarks:
    points.extend([(lm.x, lm.y, lm.z) for lm in results.face_landmarks])
else:
    points.extend([(0,0,0)] * 37)

return np.array(points)
```

## High Priority Enhancements

### 3. Implement CTC Alignment

#### Research Implementation:

```
# ctc_aligner.py
import torch
import torch.nn as nn

class SignLanguageCTCAligner(nn.Module):
    def __init__(self, num_signs, hidden_dim=256):
        super().__init__()
        self.encoder = nn.LSTM(85*3, hidden_dim, bidirectional=True)
        self.classifier = nn.Linear(hidden_dim*2, num_signs + 1) # +1 for blank
        self.ctc_loss = nn.CTCLoss(blank=0)

    def forward(self, skeletal_features):
        # skeletal_features: (30, batch, 85*3)
        encoded, _ = self.encoder(skeletal_features)
        logits = self.classifier(encoded)
        return logits

    def align(self, features, transcript):
        """Weakly supervised alignment"""
        logits = self.forward(features)
        # CTC automatically learns boundaries
        return self.ctc_loss(logits, transcript, ...)
```

## 4. Add Grammatical Reordering

### Linguistic Pipeline:

```

# linguistic_processor.py
import spacy # or stanza for Sinhala

class SinhalaToSSLReorderer:
    def __init__(self):
        # Load Sinhala POS tagger
        self.nlp = spacy.load('si_core_news_sm') # If available

    def reorder_svo_to_sov(self, text: str) -> str:
        """Convert Sinhala sentence structure to SSL signing order"""
        doc = self.nlp(text)

        # Identify grammatical components
        subject = [token for token in doc if 'subj' in token.dep_]
        verb = [token for token in doc if 'verb' in token.pos_]
        obj = [token for token in doc if 'obj' in token.dep_]

        # Reorder to SOV (sign language structure)
        reordered = subject + obj + verb

        return ' '.join([token.text for token in reordered])

```

## Medium Priority Improvements

### 5. Add Buffer Windows

```

def extract_with_buffers(self, clip_duration: float = 5.0):
    # Raw extraction window: 10-15 seconds
    RAW_WINDOW = 12.0 # Middle of 10-15 range

    # Extract raw
    raw_start = i * RAW_WINDOW
    raw_clip = self.extract_video(raw_start, RAW_WINDOW)

    # Then segment into 5s clips with buffers
    for j in range(int(RAW_WINDOW / clip_duration)):
        segment_start = raw_start + j * clip_duration - 2.0 # Pre-buffer
        segment_end = segment_start + clip_duration + 5.0 # Post-buffer
        segment_clip = raw_clip[segment_start:segment_end]

```

## 6. Add Output Normalization

```
def normalize_output(self, frame):  
    # Convert to grayscale  
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  
  
    # Contrast enhancement (CLAHE)  
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))  
    enhanced = clahe.apply(gray)  
  
    # Normalize to [0, 1]  
    normalized = enhanced / 255.0  
  
    return normalized
```

---

## Migration Path

---

### Phase 1: Quick Fixes (1-2 weeks)

1. Add lag offset parameter (backward compatible)
2. Implement pre/post buffers
3. Add normalization option

### Phase 2: Skeletal Pipeline (2-4 weeks)

1. Implement MediaPipe Holistic extraction
2. Create 85-point subset logic
3. Add K=30 uniform sampling
4. Dual output: video + skeletal

### Phase 3: Advanced Features (4-6 weeks)

1. Implement CTC alignment module
2. Add linguistic reordering

3. Validate on dataset
4. Compare video vs skeletal performance

## Conclusion

### Current System Assessment

**Strengths:**

- ✓ Production-ready audio pipeline (100% spec compliance)
- ✓ Robust PiP detection (better than static coordinates)
- ✓ Excellent ASR integration (Sinhala Whisper)
- ✓ Clean metadata architecture
- ✓ Validated with 732 real-world clips

**Your system is:**

- Suitable for general sign language recognition research
- Good for proof-of-concept and initial dataset creation
- Works well for video-based models
- Functional for basic temporal alignment studies

**The specification's approach is:**

- Designed for production SSL broadcast systems
- Optimized for background-invariant models
- Handles linguistic alignment properly
- Built for weakly supervised learning at scale

### Gap Impact Analysis

| Missing Feature   | Impact on Research                    | Workaround Available?         |
|-------------------|---------------------------------------|-------------------------------|
| Lag compensation  | <b>HIGH</b> - Breaks alignment        | ⚠ Manual post-correction      |
| Skeletal features | <b>MEDIUM</b> - Limits generalization | ✓ Video works but less robust |

| Missing Feature      | Impact on Research                       | Workaround Available?          |
|----------------------|------------------------------------------|--------------------------------|
| CTC alignment        | <b>MEDIUM</b> - Manual annotation needed | ⚠️ Can use supervised learning |
| SVO → SOV reordering | <b>LOW-MEDIUM</b> - Linguistic mismatch  | ⚠️ Models may learn implicitly |
| Buffer windows       | <b>LOW</b> - Missing prosody             | ✅ Overlap helps partially      |

### Recommendation

**For current use case (Sri Lankan Parliament dataset):**

- Your implementation is **adequate for initial research**
- Priority: Add lag compensation (critical for alignment)
- Consider: Dual pipeline (video + skeletal) for comparison

**For production SSL system:**

- Implement skeletal extraction (storage & background invariance)
- Add CTC for weakly supervised learning
- Full specification compliance recommended

### Final Score: 53% Specification Compliance

**But:** Your system has **superior PiP detection** and **perfect audio pipeline**, making it a solid foundation that can be incrementally enhanced to meet full specification requirements.

---

## Document Metadata

---

**Version:** 1.0

**Analysis Date:** April 9, 2026

**Analyzed By:** Technical Assessment Team

**Specification Source:** Technical Developer Specification: Snippet Creation Pipeline

**Implementation Source:** Voice-of-Hands GitHub Repository

**Related Documents:**

- 5-Second\_Clip\_Duration\_Strategy.md - Temporal segmentation research
  - AUDIO\_SIGN\_ALIGNMENT\_METHODODOLOGY.md - Alignment approach
  - PROJECT\_DEVELOPMENT\_TIMELINE.md - Development history
- 

**© 2026 Voice-of-Hands Research Project**